



Event-Driven Orchestration Monitoring

Streaming Airflow Metadata to Kafka via CDC



Vipin Kataria • Architect, Data/ML, Picarro Inc.

Agenda

01

Airflow & observability

What Airflow does, and why observability into it is non-negotiable

02

The problem

How teams solve this today, and why it breaks past 50 teams

03

The idea and architecture

What if? CDC turns Airflow's Postgres into an event stream

04

The build

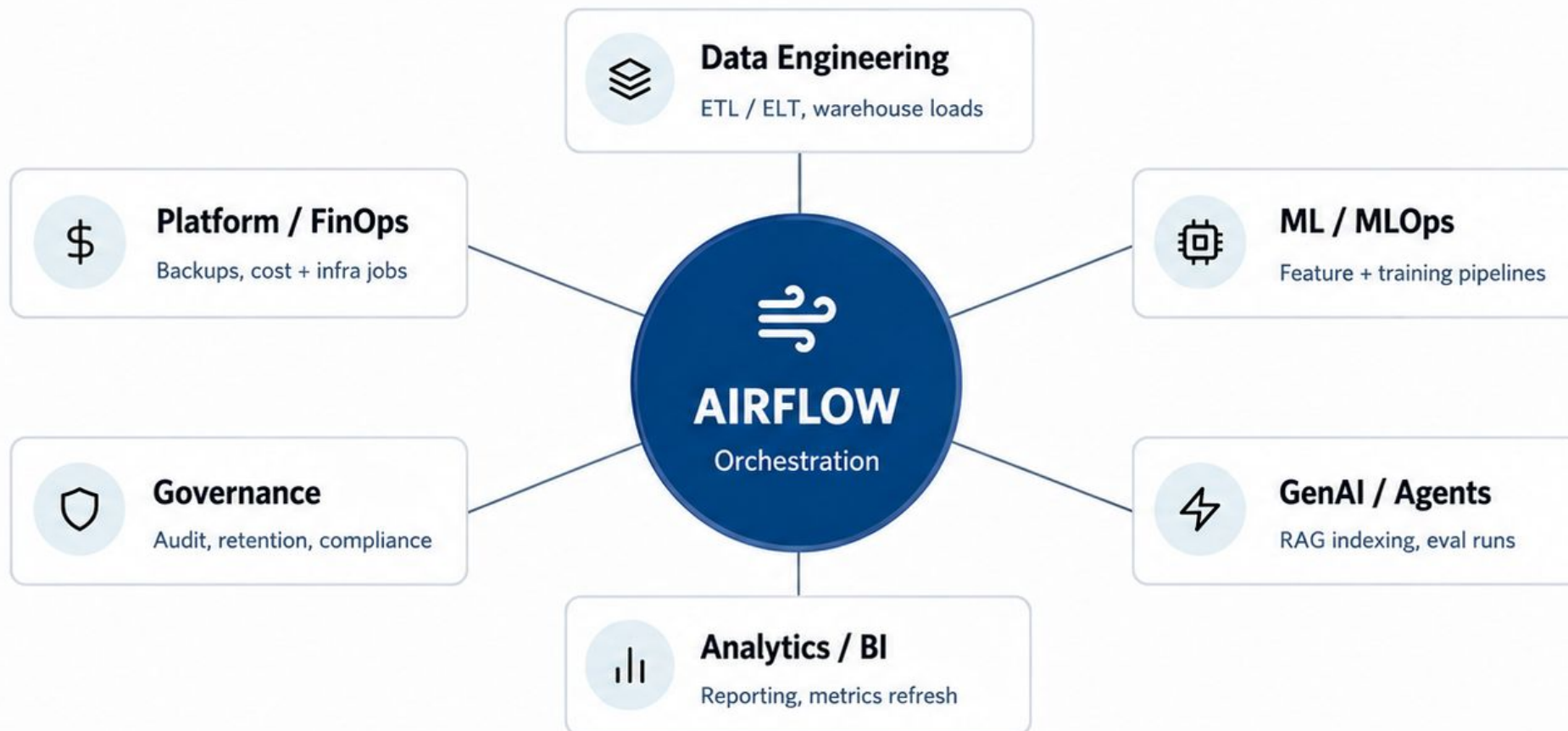
Connector setup, Kafka topic design, real-time consumers

05

Demo

— a real DAG, a real Kafka message

Airflow sits at the center of everything that runs



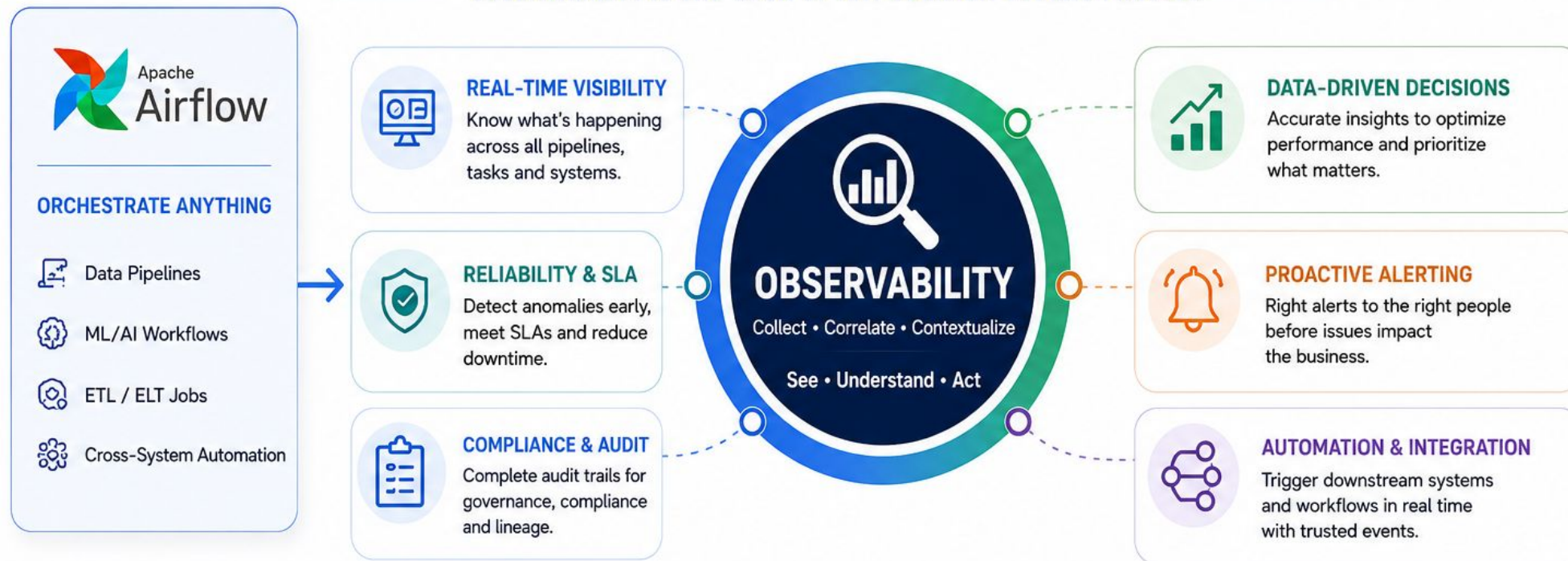
One scheduler no longer. Airflow is the control plane every workload depends on.

AIRFLOW + OBSERVABILITY: THE CRUX OF DECISION-MAKING

Airflow orchestrates your data and AI workflows.

Observability turns every state and transition into **real-time insights that drive smarter, faster decisions.**

OBSERVABILITY IS THE CRUX OF ANY DECISION-MAKING PROCESS



FROM INSIGHTS TO IMPACT



Trusted, Real-Time Data

Single source of truth for all orchestration events.



Confident Decisions

Make faster, smarter decisions with complete context.



Operational Excellence

Optimize cost, improve reliability and drive continuous improvement.



Business Impact

Faster time to insight, fewer incidents, more value delivered.

What We Observe in Airflow

Every state. Every transition. Every signal — captured for **complete visibility**.



DAG Lifecycle

End-to-end view of DAG runs from trigger to completion



What we observe

- ✓ DAG run status & outcomes
- ✓ Schedules, intervals & durations
- ✓ Dependencies & backfills



Task Lifecycle

Granular visibility into every task state and transition



What we observe

- ✓ Task instance states
- ✓ Execution time & retries
- ✓ Deferrals, dependencies & metadata



System Signals

Health and performance signals from Airflow infrastructure



What we observe

- ✓ Scheduler & worker heartbeats
- ✓ CPU / memory / queue health
- ✓ Errors, restarts & system events

Why It Matters



Complete Visibility

See the full picture across DAGs, tasks and systems.



Faster Troubleshooting

Correlate signals to quickly find root cause.



Reliability & SLA

Proactively detect issues and meet SLAs.



Audit & Compliance

Immutable records for governance and audit.



Smarter Decisions

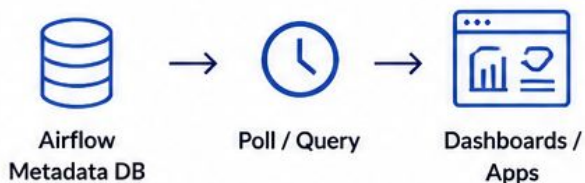
Drive automation, optimization and business impact.

How teams try to solve this today

Real-time visibility across 50 teams, without polling the DB — every org reaches for one of these first.

DIFFERENT METHODS

1 Poll-based



Continuously query Airflow's metadata DB or poll the REST API.

2 Code-embedded hooks



Add callbacks, listeners, sensors or trigger chains inside DAGs.

3 Purpose-built tools



Use external tools and integrations for monitoring and alerting.

PROS & CONS COMPARISON

Aspect	1 Poll-based	2 Code-embedded hooks	3 Purpose-built tools
✓ Pros	- Simple to implement - No code changes in DAGs	- Event-driven within Airflow - Flexible and customizable	- Rich dashboards - Great for metrics & visualization
✗ Cons	- High load on DB / API - Doesn't scale with teams - Near real-time, not real-time	- Requires code in every DAG - Retroactive coverage not guaranteed - Hard to enforce consistently	- Single-purpose by design - Lineage or metrics only - You still bolt on 2-3 more systems



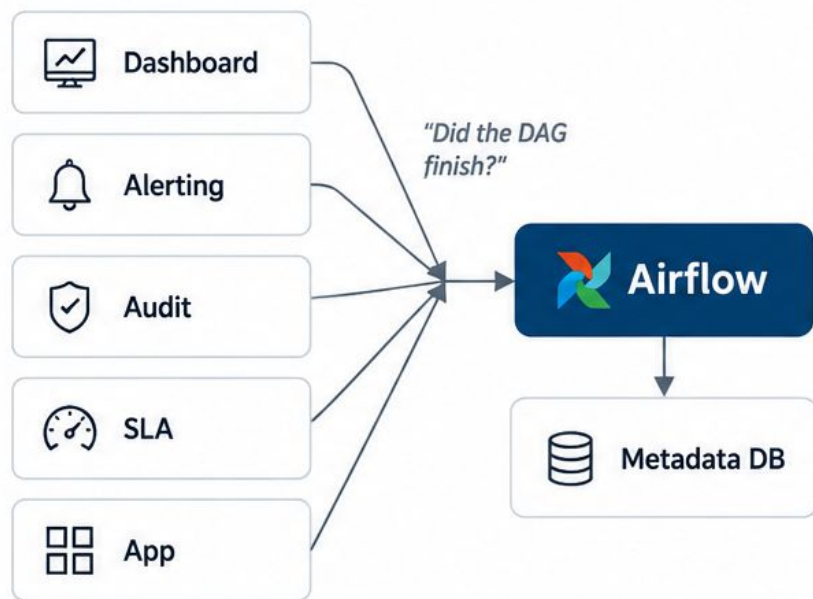
Each approach solves part of the problem — but none delivers complete, real-time, fan-out observability for 50 teams.

THE PIVOT

What if it could simply tell us when it happened?

Instead of asking Airflow, again and again, whether something happened —

TODAY



That's the idea behind Change Data Capture.

Airflow already records these events

This is what Airflow's own Postgres actually contains — real tables, real rows, captured live from the demo stack:

psql - airflow@postgres:5432/airflow - SELECT * FROM dag_run

dag_id	run_id	state	execution_date	start_date
sample_etl_dag	manual__2026-08-23T18:02:40+00:00	success	2026-08-23 18:02:40+00	2026-08-23 18:02:41.354076+00
sample_etl_dag	manual__2026-08-27T17:40:32+00:00	success	2026-08-27 17:40:32+00	2026-08-27 17:40:33.329304+00
sla_breach_demo_dag	manual__2026-08-27T17:40:44+00:00	success	2026-08-27 17:40:44+00	2026-08-27 17:40:44.334155+00

psql - airflow@postgres:5432/airflow - SELECT * FROM task_instance

dag_id	task_id	state	start_date	end_date	duration (s)
sample_etl_dag	extract	success	2026-08-27 17:40:33.43	2026-08-27 17:40:33.59	0.15
sample_etl_dag	load	success	2026-08-27 17:40:35.42	2026-08-27 17:40:38.62	3.19
sla_breach_demo_dag	fast_task	success	2026-08-27 17:40:44.42	2026-08-27 17:40:46.61	2.19
sla_breach_demo_dag	slow_task_breaches_sla	success	2026-08-27 17:40:47.54	2026-08-27 17:41:22.72	35.19

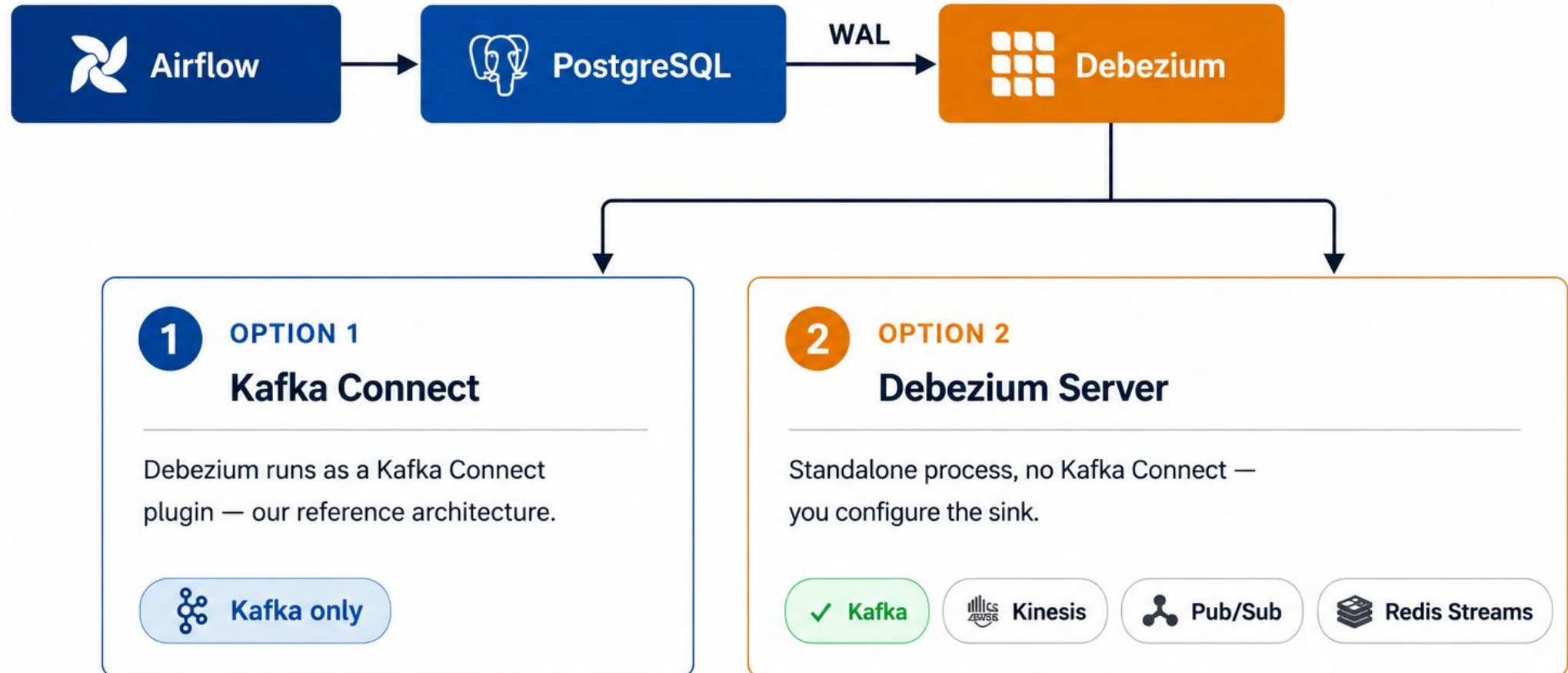


The event is already there. We don't need to recreate it.

Same Postgres your scheduler already writes to. Change *Data Capture* just reads what's already there.

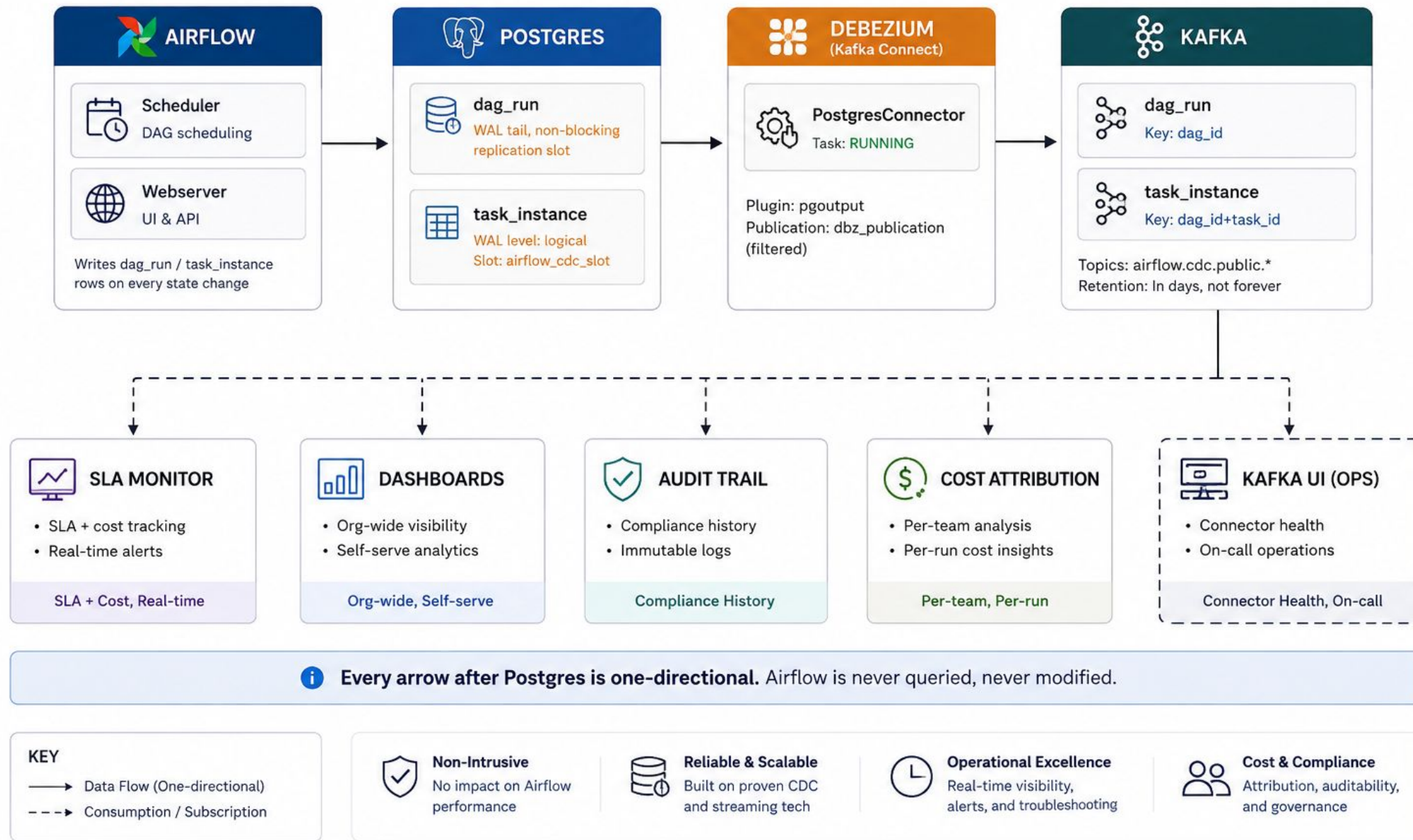
How do we capture the changes?

PostgreSQL writes changes to its WAL. Debezium tails the WAL, converts each row change into a structured event, and never queries Airflow tables. **It reads the WAL through a logical replication slot.**



Event-Driven Orchestration Monitoring – Full Stack Architecture

End-to-end visibility from Airflow events to operations and observability



Three components — two you already have

No new infrastructure category, nothing exotic. Here's the whole shopping list.

Airflow Stack

Scheduler, webserver, and its own Postgres metadata DB — dag_run, task_instance already live there.

ALREADY RUNNING

Debezium Server

One Docker image — debezium/server. Already sitting in the compose file; nothing to build.

ONE DOCKER IMAGE

Kafka + Kafka UI

Confluent's Kafka image, KRaft mode — plus Kafka UI so you can actually see the messages land.

ONE DOCKER IMAGE

Turn on one flag, bring up Debezium, bring up Kafka — everything after that happens on its own.

One file brings all three components up

Airflow, Postgres, Debezium Server, Kafka, Kafka UI, the consumer — every component from the last slide, wired together here.

```

1 x-airflow-common: &airflow-common
2 image: apache/airflow:2.9.3-python3.11
3 environment: &airflow-common-env
4 AIRFLOW__CORE__EXECUTOR: LocalExecutor
5 AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
6 AIRFLOW__CORE__LOAD_EXAMPLES: "false"
7 AIRFLOW__CORE__DAGS__PAUSED_AT_CREATION: "false"
8 AIRFLOW__WEBSERVER__EXPOSE_CONFIG: "true"
9 AIRFLOW__API__AUTH_BACKENDS: "airflow.api.auth.backend.basic_auth"
10 AIRFLOW__SCHEDULER__DAG_DIR_LIST_INTERVAL: "10"
11 volumes:
12 - ./airflow/dags:/opt/airflow/dags
13 depends_on:
14   postgres:
15     condition: service_healthy
16
17 services:
18 # --- Airflow's own metadata DB, tuned for logical replication (required by Debezium) ---
19 postgres:
20   image: postgres:15
21   command: >
22   postgres -c wal_level=logical -c max_replication_slots=10 -c max_wal_senders=10
23   environment:
24     POSTGRES_USER: airflow
25     POSTGRES_PASSWORD: airflow
26     POSTGRES_DB: airflow
27   ports:
28     - "5432:5432"
29   volumes:
30     - pg-data:/var/lib/postgresql/data
31   healthcheck:
32     test: ["CMD", "pg_isready", "-U", "airflow"]
33     interval: 5s
34     retries: 10
35
36 airflow-init:
37   <<: &airflow-common
38   entrypoint: /bin/bash
39   command:
40     - -c
41     - |
42       airflow db migrate --username admin --password admin \
43       --firstname Vipin --lastname Kataria --role Admin --email admin@example.com
44
45 airflow-webserver:
46   <<: &airflow-common
47   command: webserver
48   ports:
49     - "8080:8080"
50   depends_on:
51     - airflow-init
52
53 airflow-scheduler:
54   <<: &airflow-common
55   command: scheduler
56   depends_on:
57     - airflow-init
58
59 # --- Kafka (KRaft mode, no ZooKeeper needed) ---
60 kafka:
61   image: confluentinc/cp-kafka:7.6.0
62   environment:
63     KAFKA_NODE_ID: 1
64     KAFKA_PROCESS_ROLES: broker,controller
65     KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
66     KAFKA_LISTENERS: PLAINTEXT://kafka:9092,CONTROLLER://kafka:9093,PLAINTEXT_HOST://0.0.0.0:29092
67     KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:29092
68
69     KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
70     KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
71     KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
72     CLUSTER_ID: "Mku30EvBntcwTjDNDm20k"
73     KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
74     KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
75   ports:
76     - "29092:29092"
77   volumes:
78     - kafka-data:/var/lib/kafka/data
79
80 # --- Debezium Kafka Connect: tails Postgres WAL, publishes change events to Kafka ---
81 connect:
82   image: debezium/connect:2.7.3.Final
83   depends_on:
84     - kafka
85     - postgres
86   environment:
87     BOOTSTRAP_SERVERS: kafka:9092
88     GROUP_ID: airflow-cdc-connect
89     CONFIG_STORAGE_TOPIC: airflow_cdc_connect_configs
90     OFFSET_STORAGE_TOPIC: airflow_cdc_connect_offsets
91     STATUS_STORAGE_TOPIC: airflow_cdc_connect_status
92   ports:
93     - "8083:8083"
94
95 # --- Debezium Server: the SECOND path -- same WAL reader, but standalone, writing
96 # straight to Kafka via its own producer. No Kafka Connect anywhere in this path.
97 # Proves Debezium's WAL-tailing core doesn't require Kafka Connect to run.
98 debezium-server:
99   image: debezium/server:2.7.3.Final
100   depends_on:
101     - kafka
102     - postgres
103   volumes:
104     - ./debezium-server/conf:/debezium/conf
105     - dbz-server-data:/debezium/data
106
107 # --- Kafka UI, purely so the audience can *see* topics/messages during the demo ---
108 kafka-ui:
109   image: provectuslabs/kafka-ui:latest
110   depends_on:
111     - kafka
112     - connect
113   environment:
114     KAFKA_CLUSTERS_0_NAME: local
115     KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092
116     KAFKA_CLUSTERS_0_KAFKACONNECT_0_NAME: connect
117     KAFKA_CLUSTERS_0_KAFKACONNECT_0_ADDRESS: http://connect:8083
118   ports:
119     - "8081:8080"
120
121 # --- The real-time consumer: SLA monitor + alert console (the "downstream system") ---
122 sla-monitor:
123   build: ./consumer
124   depends_on:
125     - kafka
126     - connect
127   environment:
128     KAFKA_BOOTSTRAP_SERVERS: kafka:9092
129     SLA_BREACH_SECONDS: "20"
130
131 volumes:
132   pg-data:
133     kafka-data:
134     dbz-server-data:

```

github.com/picarro/cloud-data-platform/airflow-cdc/docker-compose.yml — git clone && docker compose up

Take one section: what is postgres trying to do here?

Seventeen lines. One of them is the entire reason this talk exists.

```
docker-compose.yml – the postgres: section (lines 18–34)

18  # --- Airflow's own metadata DB, tuned for logical replication (required by Debezium) ---
19  postgres:
20    image: postgres:15
21    command: >
22    postgres -c wal_level=logical -c max_replication_slots=10 -c max_wal_senders=10
23    environment:
24      POSTGRES_USER: airflow
25      POSTGRES_PASSWORD: airflow
26      POSTGRES_DB: airflow
27    ports:
28      - "5432:5432"
29    volumes:
30      - pg-data:/var/lib/postgresql/data
31    healthcheck:
32      test: ["CMD", "pg_isready", "-U", "airflow"]
33      interval: 5s
34      retries: 10
```

That one highlighted line is the entire prerequisite: turn on logical WAL, and Postgres starts keeping a readable log of every row change.

Now the kafka: section — what does it need to change?

Nothing. Zero lines reference Debezium, CDC, or WAL.

```
docker-compose.yml — the kafka: section (lines 60–78, trimmed)

60  # --- Kafka (KRaft mode, no ZooKeeper needed) ---
61  kafka:
62    image: confluentinc/cp-kafka:7.6.0
63    environment:
64      ... (KRaft broker/controller wiring — nothing CDC-specific)
75    ports:
76      - "29092:29092"
77    volumes:
78      - kafka-data:/var/lib/kafka/data
```

Kafka doesn't know this pipeline exists — Debezium does all the special-casing on the way in.

And kafka-ui: — just point it at the same broker

One highlighted line turns a blank UI into a live view of every topic.

```
docker-compose.yml — the kafka-ui: section (lines 107–119, trimmed)

107  # --- Kafka UI, purely so the audience can *see* topics/messages during the demo ---
108  kafka-ui:
109      image: provectuslabs/kafka-ui:latest
110      depends_on:
111          - kafka
112          ... (also depends on Kafka Connect — unused in this talk's path)
113      environment:
114          KAFKA_CLUSTERS_0_NAME: local
115          KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092
116          ... (Kafka Connect address — unused in this talk's path)
117
118      ports:
119          - "8081:8080"
```

That's the whole viewer — no separate agent, no polling script, just a bootstrap-servers address.

And debezium

```

debezium-server:
  image: debezium/server:2.7.3.Final
  depends_on:
    - kafka
    - postgres
  volumes:
    - ./debezium-server/conf:/debezium/conf
    - dbz-server-data:/debezium/data

```

```

9  debezium.sink.type=kafka
10  debezium.sink.kafka.producer.bootstrap.servers=kafka:9092
11  debezium.sink.kafka.producer.key.serializer=org.apache.kafka.common.serialization.StringSeriali
12  debezium.sink.kafka.producer.value.serializer=org.apache.kafka.common.serialization.StringSeria
13
14  debezium.source.connector.class=io.debezium.connector.postgresql.PostgresConnector
15  debezium.source.database.hostname=postgres
16  debezium.source.database.port=5432
17  debezium.source.database.user=airflow
18  debezium.source.database.password=airflow
19  debezium.source.database.dbname=airflow
20
21  # Deliberately distinct slot/publication/topic names so this runs *alongside* the
22  # Kafka Connect connector against the same Postgres without conflicting with it.
23  debezium.source.topic.prefix=airflow.cdc.server
24  debezium.source.slot.name=airflow_cdc_slot_server
25  debezium.source.publication.name=dbz_publication_server
26  debezium.source.plugin.name=pgoutput
27  debezium.source.table.include.list=public.dag_run,public.task_instance
28
29  debezium.source.offset.storage.file.filename=/debezium/data/offsets.dat
30  debezium.source.offset.flush.interval.ms=1000
31
32  quarkus.log.console.json=false
33

```

One successful run — and the message it produced

Triggered `sample_etl_dag` on this exact stack a minute ago. Here's the run, and the Kafka message it produced, expanded.

Airflow — DAG: sample_etl_dag — Grid

sample_etl_dag

SUCCESS

run_id: manual__2026-08-28T00:26:10+00:00 · triggered manually

● extract

success

0.22s

● transform

success

0.27s

● load

success

3.18s

start_date 00:26:10.579644Z → end_date 00:26:16.490855Z (total 5.9s)



Kafka UI — airflow.cdc.server.public.dag_run — offset 29, expanded

timestamp: 2026-08-28T00:26:17.055Z partition: 0 value size: 5.8 KB

```
{
  "op": "u", // update
  "before": null,
  "after": {
    "dag_id": "sample_etl_dag",
    "run_id": "manual__2026-08-28T00:26:10+00:00",
    "state": "success",
    "start_date": "2026-08-28T00:26:10.579Z",
    "end_date": "2026-08-28T00:26:16.490Z",
    "run_type": "manual"
  },
  "source": { "table": "dag_run", "connector": "postgresql" }
}
```

Same run_id. Same state. Same timestamps, down to the millisecond. That's not a coincidence — that's the WAL.

Thank you.

Questions?



Vipin Kataria

Engineering Leader | Senior IEEE Member | Ex
Intel | Ex Amazon | Machine Learning | Data...



What actually breaks at scale

Watch replication slot lag

A stalled consumer group can pin the WAL and bloat Postgres storage — alert on slot lag, not just consumer lag.

Debezium restarts replay in-flight events

Consumers must be idempotent — de-dupe on (dag_id, task_id, run_id, state) as a natural key.

Cap table.include.list deliberately

Resist the urge to CDC every Airflow table — start with dag_run/task_instance, add only on real need.

Snapshot mode matters on day one

Initial snapshot of task_instance can be huge on a mature Airflow install — schedule it off-peak.

Connector is a single point of lag, not failure

Airflow is unaffected if Connect goes down; consumers just fall behind until it's back.

Kafka UI / Connect REST for on-call

Give on-call a way to see connector + consumer-group health without reading Debezium logs.